

SmartLock

Présentation du projet :

- *Le projet SmartLock modernise la gestion du matériel du DeVinci Fablab en remplaçant les serrures traditionnelles par un système d'accès sécurisé et connecté via badge NFC.*
- *Cette solution couple la sécurisation physique des armoires à une gestion centralisée des stocks pour garantir une traçabilité complète des emprunts en temps réel.*
- *A terme, la solution sera implémentée toutes les armoires du Fablab afin de sécuriser le matériel de l'espace.*

V3 - VF	Description et résultats des tests sous-systèmes	17/05/2026
Rédacteur : Equipe	Relecteurs : Florian Cazac, Guilhem Henoux, Alexandre Houdin, Augustin Winterbert, Lois Halmaert, Yann Videmment	Jalon revue des tests
	Validateurs : Florian Cazac	





Table des matières

Introduction.....	3
1.1. Architecture générale du système	3
2. Sous-système : PCB custom — Assemblage et soudure.....	3
2.1. Description	3
2.2. Procédure.....	4
2.3. Résultats et observations.....	4
3. Sous-système : Régulateurs DC-DC (MC34063ADR)	5
3.1. Description	5
3.2. Configurations des 4 régulateurs	5
3.3. Problème identifié : limitation du courant de sortie	5
3.4. Correction prévue.....	5
4. Sous-système : BMS 3S et pack batteries Li-Ion.....	6
4.1. Description	6
4.2. Résultats et observations	6
5. Sous-système : Intégration mécanique.....	7
5.1. Description	7
5.2. Résultats et observations.....	7
6. Sous-système : API et Database.....	8
6.1. Description	8
6.2. Architecture et routes principales.....	8
6.3. Tests réalisés.....	9
6.4. Résultats et observations	10
7. Sous-système : Interface Homme-Machine (IHM)	11
7.1. Description	11
7.2. Architecture des vues	11
7.3. Intégration API REST et lecture NFC	11
7.4. Résultats	12
8. Bilan et prochaines étapes.....	13
8.1. Synthèse	13
8.2. Actions immédiates	13



8.3.	Améliorations prévues.....	13
------	----------------------------	----



1. Introduction

Ce document présente les descriptions et résultats des tests réalisés sur les différents sous-systèmes du projet SmartLock. SmartLock est une serrure connectée et autonome destinée à sécuriser les armoires de l'association DVFL et à gérer une base de données facilitant la gestion des stocks. Le système embarque une interface utilisateur (écran tactile + RFID), un microcontrôleur (Raspberry Pi Zero 2W), un verrou électromagnétique (solénoïde 12V) et une chaîne d'alimentation complète incluant une batterie de secours 3S Li-Ion avec BMS.

1.1. Architecture générale du système

Le PCB custom (version V4) intègre les sous-systèmes suivants, d'après le schéma KiCad Smartlock_archi_electro_V4 :

- Alimentation secteur : adaptateur AC/DC 220V → 12V (J8)
- 4 régulateurs DC-DC à base de MC34063ADR (U1 à U4) : 12V→5V, 11,1V→5V, 11,1V→12V, 12V→12,6V
- BMS 3S (J10) + pack 3×18650 Samsung 25R pour l'alimentation de secours
- Raspberry Pi Zero 2W (U10) - cerveau du système
- Lecteur RFID/NFC PN532 (I²C via GPIO2/SDA, GPIO3/SCL)
- Écran 7" HDMI capacitif Waveshare 1024×600 (U20)
- Solénoïde 12V (L10) - actionneur du verrou
- Capteur à effet Hall A1101xLH (U15) - détection position verrou
- Module BIGTREETECH SKSM V1.0 (J2) - sauvegarde données
- LEDs de signalisation (D20, D21), buzzer (BZ2), interrupteur de protection (SW1), fusible 3,15A (F1)
- MOSFET N-channel (Q1) - commande du solénoïde
- Optocoupleur LTV-817 (U5) - isolation galvanique

2. Sous-système : PCB custom - Assemblage et soudure

Responsable : Guilhem Henoux

2.1. Description

Après conception du schéma électronique sous KiCad et routage du PCB, la carte a été assemblée manuellement. Il s'agissait d'un premier assemblage SMD (Surface Mount Device), impliquant un apprentissage progressif des techniques de soudure fine. Les composants concernés sont les régulateurs MC34063ADR, condensateurs, résistances et inductances CMS (0402/0603/0805) ainsi que les diodes Schottky 1N5822. La durée estimée a été de 6 à 10 heures, principalement en raison de la courbe d'apprentissage.

2.2. Procédure

- Application de pâte à souder sur les pastilles CMS
- Placement des composants SMD et passage sur plaque chauffante
- Soudure des composants traversants (JST)
- Inspection visuelle et contrôle de continuité au multimètre

Pour certains composants plus grands comme les diodes, la soudure a été réalisée directement à l'étain sans pâte à souder.

2.3. Résultats et observations

Le PCB est fonctionnel : aucun court-circuit, toutes les pistes sont continues, et l'intégration est conforme au layout KiCad. Un défaut mineur subsiste — un câble devra relier le pin 12V au pin 4 de l'optocoupleur. Des soudures plus robustes (remplacement des headers par des soudures directes) sont prévues en fin de projet.

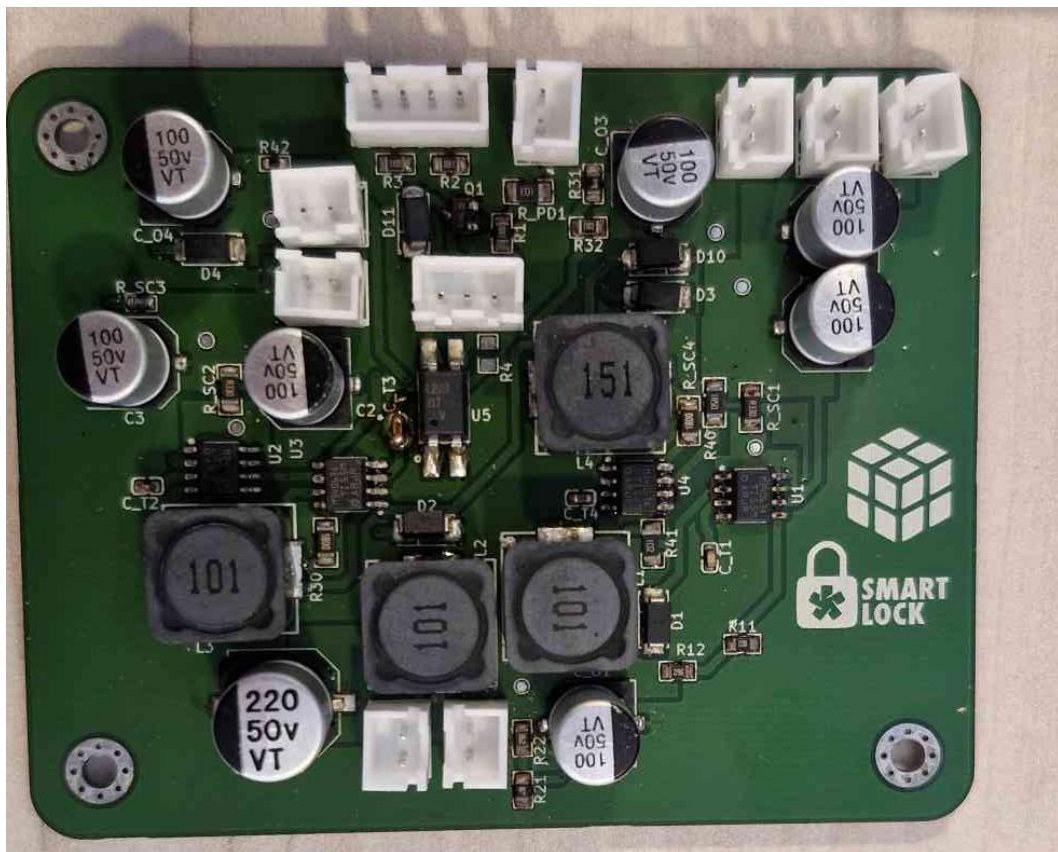


Figure 1 PCB assemblé



3. Sous-système : Régulateurs DC-DC (MC34063ADR)

Responsable : Guilhem Henoux

3.1. Description

Le PCB embarque quatre régulateurs DC-DC basés sur le MC34063ADR (U1 à U4), un convertisseur DC-DC monolithique capable de fonctionner en step-down, step-up ou inverting. Chaque régulateur est dimensionné pour une tension de sortie spécifique, calculée par le pont diviseur résistif R1/R2 et les composants passifs associés.

3.2. Configurations des 4 régulateurs

Le PCB embarque quatre régulateurs aux rôles distincts. U1 et U2 sont tous deux configurés en mode buck avec une sortie 5V à ~1A : U1 est alimenté depuis le secteur (12V) et alimente la Raspberry Pi et les périphériques en fonctionnement normal, tandis que U2 prend le relais depuis la batterie (11,1V) en cas de coupure secteur. U3 est configuré en boost depuis la batterie (11,1V → 12V, ~0,6A) pour alimenter le solénoïde en mode secours. Enfin, U4 est un boost dédié à la charge du pack batteries (12V → 12,6V, ~250mA).

3.3. Problème identifié : limitation du courant de sortie

Lors des tests avec la Raspberry Pi 4, une chute de tension significative a été mesurée à la connexion de l'écran : 4,94V sans charge, 4,82V sous charge Raspberry Pi 4, et l'écran ne s'allumait plus. Avec la Raspberry Pi Zero 2W, l'écran s'allume faiblement mais ne démarre pas.

La cause identifiée est la résistance R_SC, dimensionnée pour 1A maximum. La Raspberry Pi Zero 2W consomme jusqu'à 700mA sous charge, et l'écran Waveshare 7" nécessite environ 500mA supplémentaires, soit ~1,2A total, au-delà de la capacité actuelle. Des pertes résistives dans les câbles de test (section insuffisante) aggravaient également la chute de tension mesurée.

3.4. Correction prévue

Remplacement des 4 résistances R_SC (R_SC1 à R_SC4) pour doubler le courant de sortie autorisé (~2A), avec ajustement de deux résistances en sortie de régulateur pour corriger légèrement la tension. Les nouvelles valeurs sont en cours d'approvisionnement (Mouser/LCSC). Les valeurs recherchées n'étaient pas disponibles en stock DVB ou IFT lors de la recherche du 06/05/2026.

4. Sous-système : BMS 3S et pack batteries Li-Ion

Responsable : Guilhem Henoux

4.1. Description

Le système embarque une batterie de secours composée de 3 cellules 18650 Samsung INR25R (2500mAh, 20A max en décharge) montées en série, formant un pack 3S (tension nominale 11,1V, tension max 12,6V). La gestion est assurée par un BMS 3S (J10, Fasizi) qui protège les cellules contre la surcharge, la sous-tension et les courts-circuits, et assure l'équilibrage. La charge s'effectue via le régulateur U4 (12V → 12,6V, ~250mA) et la décharge via les régulateurs U2 (→5V) et U3 (→12V).

4.2. Résultats et observations

Le BMS a été soudé et testé de manière isolée avec succès : fonctionnement confirmé en charge (tension cellules < 4,2V) et en décharge à petit courant. L'intégration complète au circuit PCB est prévue après correction du sous-système régulateurs (remplacement des R_SC). Cette précaution est volontaire : intégrer un pack batteries dans un circuit non encore stabilisé en courant présente des risques, et il est préférable de procéder méthodiquement.

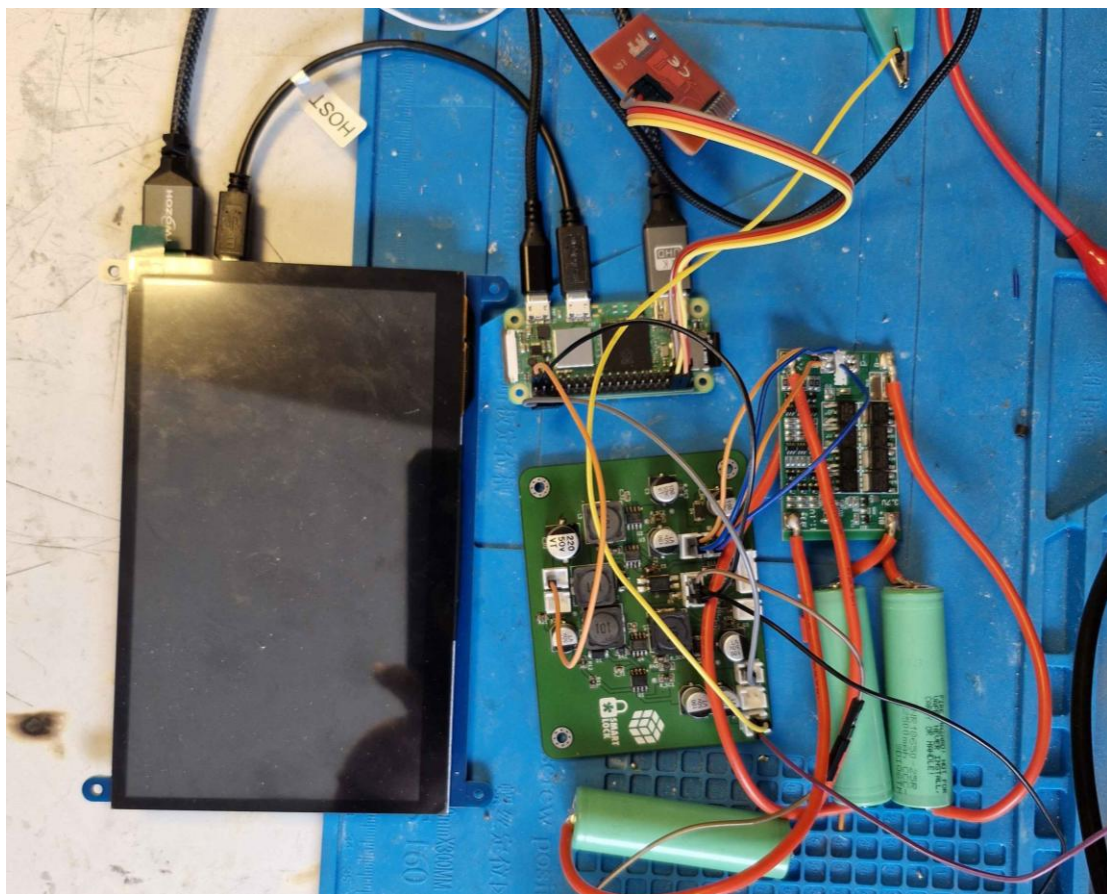


Figure 2 Test de BMS

5. Sous-système : Intégration mécanique

Responsable : Alexandre Houdin

5.1. Description

Cette phase couvre l'intégration physique de l'ensemble des composants (PCB, écran, solénoïde, câbles, BMS, pack batteries) dans le boîtier mécanique du SmartLock. L'objectif est de vérifier que la modélisation 3D permet d'accueillir tous les éléments sans contrainte, et d'identifier les ajustements nécessaires.

5.2. Résultats et observations

Le test d'intégration a permis de valider le principe d'assemblage général. Cependant, plusieurs points de la modélisation 3D devront être modifiés pour accommoder l'ensemble des câbles et des composants dans les contraintes réelles du boîtier. Les zones de conflit identifiées sont en cours d'intégration dans le modèle CAO.

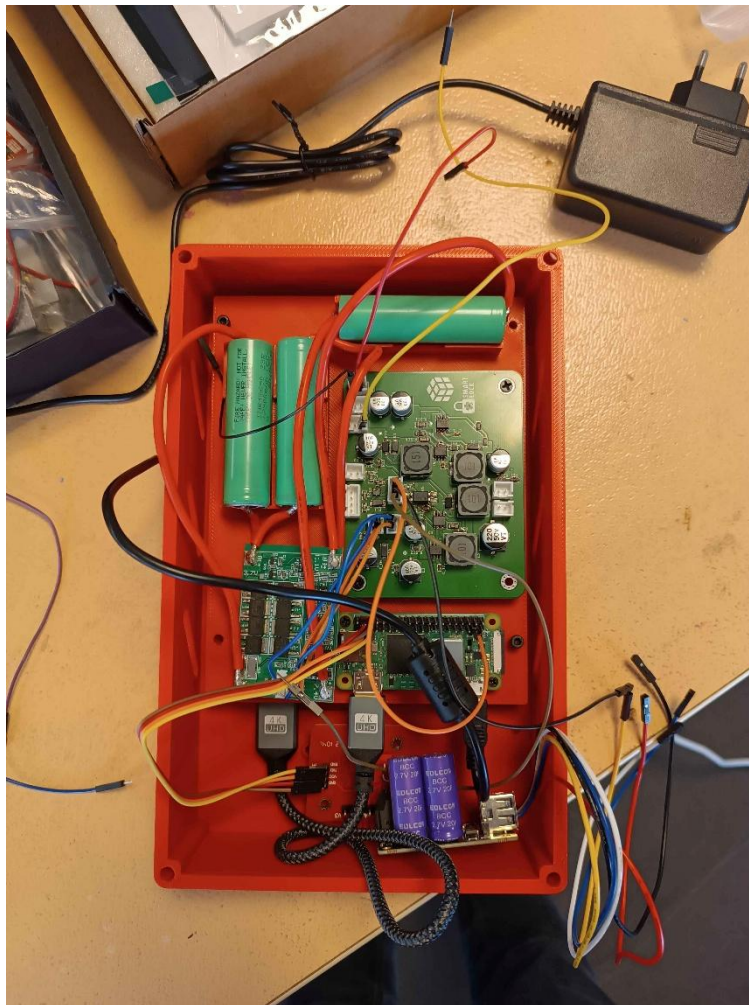


Figure 3 Tests d'intégration électronique/mécanique



6. Sous-système : API et Database

Responsable : Florian Cazac

6.1. Description

L'API REST constitue le cœur logiciel du système SmartLock. Développée avec le framework FastAPI (Python 3.12), elle assure la gestion centralisée des accès aux casiers, de l'inventaire du stock, et la traçabilité de l'ensemble des tentatives d'accès. Elle s'interface avec Keycloak (serveur d'identité open-source) pour l'authentification par jetons JWT et la gestion des rôles utilisateurs, et s'appuie sur une base de données PostgreSQL pour la persistance des données.

L'API sert deux types de clients :

Le tableau de bord web (Svelte SPA) : interface d'administration pour la gestion des casiers, des permissions, des utilisateurs et du stock

Le terminal Raspberry Pi : vérification d'accès en temps réel lors d'un badgeage NFC

Le déploiement est conteneurisé via Docker Compose, avec exécution automatique des migrations de base de données au démarrage.

6.2. Architecture et routes principales

L'API est structurée en plusieurs modules fonctionnels indépendants :

Module	Route	Rôle
Authentification NFC	/auth	Vérification d'accès carte → casier
Casiers	/lockers	CRUD de l'inventaire des casiers
Permissions	/permissions	Association rôle → casier avec niveau d'accès
Utilisateurs	/users	Proxy Keycloak + cycle de vie des comptes
Rôles	/roles	Gestion des rôles avec hiérarchie de niveaux
Badges NFC	/badge	Enregistrement et association des cartes NFC
Logs d'accès	/logs	Journal d'audit des tentatives d'accès
Stock	/items, /categories, /stock	Gestion du catalogue et des niveaux de stock



Flux principal de vérification d'accès (POST /auth/locker/{id}/check) :

Le terminal Raspberry Pi envoie l'identifiant brut de la carte NFC à l'API
L'API hache l'identifiant (SHA-256) et interroge Keycloak pour identifier l'utilisateur correspondant
L'API vérifie que le compte utilisateur est actif (non révoqué)
Les rôles effectifs de l'utilisateur sont récupérés depuis Keycloak
La base de données est interrogée pour déterminer si un rôle de l'utilisateur dispose d'une permission sur le casier demandé
L'API retourne une décision (allowed : true/false) avec le niveau de permission effectif (can_view, can_open, can_edit)
La tentative d'accès (accordée ou refusée, avec motif) est enregistrée dans le journal d'audit

Modèle de permission : Les permissions sont stockées dans la table locker_permissions sous la forme d'une association rôle → casier → niveau. La hiérarchie can_view < can_open < can_edit est résolue dynamiquement. Les permissions peuvent être limitées dans le temps via un champ valid_until.

6.3. Tests réalisés

Les tests ont été conduits sur trois niveaux.

Tests automatisés (pytest) - 87 cas de test - Voir Figure

Fichier de test	Cas	Couverture
test_routes.py	55	Endpoints REST, gardes RBAC, codes HTTP
test_roles_crud.py	9	CRUD des rôles, hiérarchie de niveaux
test_crud_role.py	9	Opérations base de données sur les rôles
test_users_lifecycle.py	7	Révocation, restauration, suppression de compte
test_keycloak_admin.py	3	Client Keycloak asynchrone (appels simulés)
test_roles_model.py	2	Modèle de rôle (champs, unicité)
test_permission_model.py	2	Modèle de permission (champs, contrainte unique)

Infrastructure de test : base SQLite en mémoire par test, appels Keycloak simulés via AsyncMock, limitation de débit désactivée, fixtures de clients par rôle (admin, codir, membre, nfc-scanner, raspberry pi).



Tests d'intégration Keycloak

Validation JWT avec vérification de signature via JWKS (clés publiques Keycloak)
Service accounts testés avec grant client_credentials : terminal Raspberry Pi (smartlock-lockers) et scanner NFC (nfc-scanner)
Hiérarchie des rôles, contraintes de niveau et auto-protection (impossibilité de se révoquer soi-même) validées

Tests end-to-end avec le Raspberry Pi

Badgeage NFC réel → envoi à l'API → décision → commande relais → ouverture du casier
Scénarios de refus testés : carte inconnue, compte révoqué, absence de permission, permission expirée
Journal d'audit vérifié après chaque scénario

6.4. Résultats et observations

87/87 tests automatisés passants, aucun échec
L'API est opérationnelle et connectée à l'IHM du terminal Raspberry Pi
Le flux NFC complet (badgeage → décision → ouverture) fonctionne en conditions réelles
La sécurité repose sur le hachage de l'identifiant NFC côté serveur : aucun secret ne transite entre le terminal et l'API
Un mécanisme de mise en cache des tokens de service account (30 secondes) limite la charge sur Keycloak lors de badgeages fréquents

Voir Annexe



7. Sous-système : Interface Homme-Machine (IHM)

Responsable : Loïs Halmaert

7.1. Description

L'IHM est développée en Python avec la librairie CustomTkinter et fonctionne sur un Raspberry Pi 4 équipé d'un écran tactile LCD 1024×600. Elle gère l'ensemble des interactions utilisateur : badgeage NFC, navigation dans les stocks, sélection d'articles, validation et clôture de session. L'interface est entièrement responsive — toutes les dimensions sont calculées dynamiquement à partir de la résolution cible (g.SW = 1024, g.SH = 600) — et un design system cohérent a été appliqué sur l'ensemble des vues (corner_radius uniforme, palette de couleurs unifiée, typographies cohérentes).

7.2. Architecture des vues

- home_view.py : écran d'accueil avec badgeage NFC automatique
- navigation_view.py : menu principal (Tendances, Filaments, Électronique)
- selection_couleur.py : choix de la couleur pour les filaments
- ecran_selection.py : sélection de la quantité avec affichage du stock
- vue_panier.py : récapitulatif des articles sélectionnés
- validation_finale.py : confirmation de la commande
- acces_physique.py : écran d'ouverture de l'armoire
- ecran_cloture.py : bilan de session avec signalement de problèmes

Un module timer_manager.py centralise la gestion de l'inactivité : 30 secondes sur l'écran d'accueil entraînent la fermeture du programme, 90 secondes sur les autres écrans un retour à l'accueil, et 10 minutes sur l'écran d'accès physique déclenchent une alerte Discord.

7.3. Intégration API REST et lecture NFC

L'IHM est connectée à l'API REST SmartLock (api.smartlock.devinci-fablab.fr), sécurisée par Keycloak. Les fonctionnalités implémentées couvrent l'authentification via client_credentials avec renouvellement automatique du token, l'identification badge (POST /auth/locker/{id}/check), la récupération des stocks (GET /lockers/{id}/stock), l'enregistrement des transactions (PUT /stock/{id}), la commande GPIO directe pour l'ouverture du relais, et la détection de fermeture de porte par polling GPIO toutes les 2 secondes.

La lecture NFC est automatique : un thread démarre en arrière-plan dès l'affichage de l'écran d'accueil et attend le badge sans intervention de l'utilisateur. Un flag SIMULATION_MODE permet de basculer entre mode simulé (données fictives, pas de réseau) et mode réel sans modifier la logique de l'IHM.



7.4. Résultats

Les tests en mode réel ont confirmé le bon fonctionnement de l'ensemble : démarrage de l'IHM sur Raspberry Pi validé, responsive vérifié sur 6 résolutions différentes, authentification Keycloak fonctionnelle avec renouvellement automatique, badge NFC physique détecté et transmis à l'API, stocks chargés depuis la base de données réelle, transactions enregistrées correctement, et timers d'inactivité opérationnels. Le déploiement via SSH/scp a nécessité des corrections spécifiques à l'environnement Pi (rendu des emojis, affichage des images, lancement automatique au démarrage).



8. Bilan et prochaines étapes

8.1. Synthèse

À ce stade du projet, trois sous-systèmes sont entièrement validés. Le PCB est fonctionnel, les soudures ont été vérifiées et l'IHM est opérationnelle et connectée à l'API et au lecteur NFC. Quatre sous-systèmes sont en cours de finalisation : les régulateurs DC-DC (remplacement des R_SC prévu), le BMS et le pack batteries (testés isolément, intégration PCB à faire), l'écran HDMI 7" (bloqué par la limitation de courant des régulateurs) et l'intégration mécanique (assemblage validé, modélisation CAO à ajuster). Les sous-systèmes restants — lecteur RFID PN532, capteur Hall, solénoïde + MOSFET et module SKSM — n'ont pas encore été testés sur le PCB complet et constituent les prochaines priorités, le solénoïde étant le plus critique.

8.2. Actions immédiates

- Commander les résistances R_SC de remplacement (Mouser/LCSC) pour passage à ~2A
- Remplacer les 4 résistances R_SC1, R_SC2, R_SC3, R_SC4 sur le PCB
- Intégrer le BMS au circuit PCB et tester le système complet en charge
- Mettre à jour la modélisation CAO suite aux modifications d'intégration mécanique identifiées
- Tester le démarrage complet Raspberry Pi + écran après correction des régulateurs

8.3. Améliorations prévues

- Remplacement des headers par des soudures directes pour améliorer la robustesse des connexions
- Utilisation de câbles de section suffisante pour limiter les pertes résistives
- Repositionnement définitif de l'écran et du solénoïde dans le boîtier

9. Annexes

Figure 4 Resultats des tests de l'API



Carte des routes - API SmartLock v0.1.0

System		
GET	/	Point d'entree principal
GET	/health	Verification etat du serveur
Authentification hardware		
POST	/auth/locker/{locker_id}/check	Verification NFC - controle d'accès au casier
Badge (scan NFC)		
POST	/badge/scan	Enregistrer un scan NFC (service NFC)
GET	/badge/pending	Cartes en attente d'assignation (admin)
PATCH	/badge/{card_id}/assign	Marquer une carte comme assignee (admin)
Categories		
GET	/categories/	Lister les categories
POST	/categories/	Creer une categorie (admin)
GET	/categories/{id}	Detail d'une categorie
PUT	/categories/{id}	Modifier une categorie (admin)
DELETE	/categories/{id}	Supprimer une categorie (admin)
Articles (Items)		
GET	/items/	Lister les articles
POST	/items/	Creer un article (admin)
GET	/items/{id}	Detail d'un article
PUT	/items/{id}	Modifier un article (admin)
DELETE	/items/{id}	Supprimer un article (admin)
Casiers (Lockers)		
GET	/lockers/	Lister les casiers
POST	/lockers/	Creer un casier (admin)
GET	/lockers/{id}	Detail d'un casier
PUT	/lockers/{id}	Modifier un casier (admin)
DELETE	/lockers/{id}	Supprimer un casier (admin)
GET	/lockers/{id}/stock	Inventaire d'un casier
Permissions casier		
POST	/lockers/{id}/permissions	Creer une permission (admin)
GET	/lockers/{id}/permissions	Lister les permissions d'un casier
PUT	/lockers/permissions/{id}	Modifier une permission (admin)
DELETE	/lockers/permissions/{id}	Supprimer une permission (admin)
Gestion des roles		
GET	/roles	Lister les roles
POST	/roles	Creer un role personnalise (role_admin)
PUT	/roles/{role_name}	Modifier un role (role_admin)
DELETE	/roles/{role_name}	Supprimer un role (role_admin)
Attribution de roles		
POST	/users/{user_id}/roles/{role_name}	Attribuer un role a un utilisateur
DELETE	/users/{user_id}/roles/{role_name}	Revoquer un role d'un utilisateur
Stock		
GET	/stock/	Lister le stock
POST	/stock/	Creer une entree de stock (admin)
GET	/stock/{id}	Detail d'une entree de stock
PUT	/stock/{id}	Modifier une entree (admin)
DELETE	/stock/{id}	Supprimer une entree (admin)
Utilisateurs (Keycloak)		
GET	/users	Lister les utilisateurs (admin)
GET	/users/{id}	Detail d'un utilisateur (admin)
GET	/users/{id}/roles	Roles d'un utilisateur (admin)
GET	/groups	Groupes Keycloak (admin)
Cycle de vie utilisateur		
POST	/users/{id}/revoke	Revoquer un compte (lifecycle_manager)
POST	/users/{id}/restore	Restaurer un compte (lifecycle_manager)
DELETE	/users/{id}	Supprimer un utilisateur (lifecycle_admin)
Journal d'audit		
GET	/logs/	Logs d'accès pages (admin / codir)

Legende : GET POST PUT DELETE PATCH

Figure 5 Listes des endpoints